

Sequelize vs Native SQL Query Comparison Chart

Operation	Sequelize (ORM)	Native SQL
 Connect to DB	<pre>const sequelize = new Sequelize('postgres://user:pass@localhost:5432/db');</pre>	<pre>const pool = new Pool();</pre>
 Create a record	<pre>Model.create({ name: 'Satish' })</pre>	<pre>INSERT INTO users (name) VALUES ('Satish');</pre>
 Find all	<pre>Model.findAll()</pre>	<pre>SELECT * FROM users;</pre>
 Find one	<pre>Model.findOne({ where: { email: 'a@b.com' } })</pre>	<pre>SELECT * FROM users WHERE email = 'a@b.com';</pre>
 Find by ID	<pre>Model.findById(id)</pre>	<pre>SELECT * FROM users WHERE id = 1;</pre>
 Update one	<pre>Model.update({ name: 'Ram' }, { where: { id: 1 } })</pre>	<pre>UPDATE users SET name = 'Ram' WHERE id = 1;</pre>
 Update by ID	<pre>Model.update({ name: 'Ram' }, { where: { id } })</pre>	<pre>UPDATE users SET name = 'Ram' WHERE id = 1;</pre>
 Delete one	<pre>Model.destroy({ where: { id: 1 } })</pre>	<pre>DELETE FROM users WHERE id = 1;</pre>
 Delete by ID	<pre>Model.destroy({ where: { id } })</pre>	<pre>DELETE FROM users WHERE id = 1;</pre>

Operation	Sequelize (ORM)	Native SQL
 Join (Associations)	User.hasMany(Blog); Blog.belongsTo(User);	SELECT * FROM users INNER JOIN blogs ON users.id = blogs.userId;
 Count records	Model.count()	SELECT COUNT(*) FROM users;
 Aggregation	Model.aggregate([...])	SELECT COUNT(*) FROM users WHERE createdAt > '2022-01-01';
 Validation	{ name: { type: DataTypes.STRING, allowNull: false } }	✗ Must be handled in code
 Middleware/Hook	Model.beforeCreate(fn)	✗ Must be handled in code
 Schema	Defined through Model and DataTypes	✗ Not defined (requires custom code)
 Model definition	const User = sequelize.define('User', { name: DataTypes.STRING });	✗ Directly in queries with CREATE TABLE
 Raw Query Execution	sequelize.query('SELECT * FROM users')	db.query('SELECT * FROM users');
 Transactions	sequelize.transaction(async (t) => { ... })	BEGIN; ... COMMIT;

Example Use Case Comparison

Sequelize ORM - Finding a User by Email

js

CopyEdit

```
const user = await User.findOne({ where: { email: 'test@example.com' } });
```

✅ Native SQL - Finding a User by Email

js

CopyEdit

```
const result = await pool.query('SELECT * FROM users WHERE email = $1',  
['test@example.com']);
```

```
const user = result.rows[0];
```

🔗 Another Example: Create a Blog

✅ Sequelize ORM - Create a Blog

js

CopyEdit

```
const blog = await Blog.create({  
  title: 'My New Blog',  
  content: 'This is a new blog post.',  
  authorId: userId  
});
```

✅ Native SQL - Create a Blog

js

CopyEdit

```
const result = await pool.query(  
  'INSERT INTO blogs (title, content, author_id) VALUES ($1, $2, $3) RETURNING *',  
  ['My New Blog', 'This is a new blog post.', userId]  
);  
  
const blog = result.rows[0];
```

😬 Which One to Use?

Project Type	Pick
Simple Projects with less DB logic	☒ Native SQL
Complex projects with models/relations	☒ Sequelize ORM
Rapid Development	☒ Sequelize ORM
Data-Intensive and Performance-Critical	☒ Native SQL